

Make Specification — Best Invest Properties

Date: 25 September 2026

Orchestrator: Make

Source of truth: Bubble

Division of responsibility: Make **collects** data, Bubble **stores and calculates**, OpenAI **explains**. No Make scenario writes a published financial figure.

1. Role of Make in the system

Make is used for long-running, external and repeatable processes:

- **collecting comparable rental listings** from approved portals and market-data providers;
- generating the AI narrative via OpenAI;
- transactional email and status notifications;
- delivering introductions after the admin decision;
- digests of new matches and price/availability alerts;
- notifying the team about failures;
- future CRM synchronisation.

Make must **not**:

- be the primary business database;
- decide score, yield, tax or approval on its own;
- hold a Bubble admin token with unrestricted access when a narrow Workflow API endpoint can do the job;
- change published records directly without server-side validation in Bubble;
- make any final legal or investment decision.

2. Integration overview

```
sequenceDiagram
    participant B as Bubble backend
    participant M as Make webhook
    participant O as OpenAI / Email provider
    participant A as Bubble callback

    B->>B: Create Integration Job (queued)
    B->>M: POST job_id + idempotency_key
    M->>B: GET scoped job payload
    B-->>M: minimal validated payload
    M->>O: external action
    O-->>M: result / provider id
    M->>A: POST completion callback
    A->>A: validate status + idempotency + apply result
    A-->>M: 200 already_applied | applied
```

Bubble creates the Integration Job **before** calling Make. The webhook carries the minimum data; Make fetches the full scoped payload with a separate authenticated request. This reduces PII in Make queues/logs and prevents entity data from being spoofed through the webhook body.

3. Environments and connections

Separate Make Teams/folders, or at least separate connections, for:

- BIP - Development
- BIP - Production

Connections/secrets:

Name	Where stored	Rule
Bubble Workflow API bearer token	Make connection/secret	separate Dev/Live; rotate quarterly or after an incident
Make custom webhook API key	Bubble API Connector private header	X-Make-Apikey ; never in URL/Option Set
OpenAI API key	Make OpenAI/HTTP connection	production project key with budget/rate limits
Email provider API key	Make connection	separate sending domain/environment
Callback shared secret	Make secret + Bubble API Connector/private config	separate Dev/Live

Bubble outgoing calls must use private header parameters. The Bubble API Connector keeps private keys server-side; Development and Live keys are set separately.

4. Common webhook envelope

Request Bubble → Make

```
{
  "event_version": "1.0",
  "event_type": "ai.analysis.requested",
  "job_id": "job_01J...",
  "entity_type": "unit",
  "entity_id": "unt_01J...",
  "idempotency_key": "ai-analysis:unt_01J:score-v12:prompt-v1",
  "occurred_at": "2026-09-17T12:00:00Z",
  "environment": "live"
}
```

The idempotency key includes the prompt version. When the analysis prompt gets a new version, previous keys automatically become invalid and regeneration happens correctly without manual cleanup.

Headers:

```
Content-Type: application/json
X-Make-Apikey: <private>
X-Correlation-Id: <job_id>
```

Never send in the envelope: email, phone, documents, exact address, OpenAI key, Bubble admin key.

Immediate Make response

```
{
  "accepted": true,
  "job_id": "job_01J..."
}
```

The Make webhook must respond quickly. For long scenarios, do not wait for the result in a Bubble page workflow; the UI reads the Integration Job status.

Callback Make → Bubble

```
{
  "event_version": "1.0",
  "job_id": "job_01J...",
  "idempotency_key": "ai-analysis:unt_01J:score-v12:prompt-v1",
  "status": "succeeded",
  "provider_request_id": "resp...",
  "result": {},
  "metrics": {
    "duration_ms": 4820,
    "input_tokens": 1850,
    "output_tokens": 620
  },
  "completed_at": "2026-09-17T12:00:05Z"
}
```

Bubble callback workflow:

1. checks the bearer/shared secret;
2. finds the Integration Job by `job_id`;
3. checks `idempotency_key`, job type and the allowed status transition;
4. if the job is already `succeeded`, returns `200 {"result":"already_applied"}`;
5. validates result fields and the current related records;
6. writes the business result, the Audit Event and `succeeded` in one backend flow;
7. returns `200 {"result":"applied"}`.

5. Scenario register

ID	Scenario	Trigger	Priority	MVP
MK-01	Generate investment analysis	instant webhook	high	yes
MK-02	Transactional email dispatcher	instant webhook	high	yes
MK-03	Approved introduction delivery	instant webhook	critical	yes
MK-04	Developer/application decision notification	instant webhook	high	yes
MK-05	Price and availability alerts	instant webhook	medium	yes
MK-06	Saved-search match digest	scheduled	medium	yes
MK-07	Integration dead-letter alert	scheduled/instant	critical	yes
MK-08	CRM export	instant webhook	low	after MVP
MK-09	Bulk re-score progress relay	instant webhook	medium	yes
MK-10	Comparable rental data collection	scheduled	high	yes

6. Scenario details

MK-01 Generate investment analysis

Trigger: Integration Job `job_type = ai_analysis_generate`.

Modules/steps:

1. Webhooks — Custom webhook with API key.
2. JSON parse + schema/version check.
3. HTTP GET/POST to the Bubble endpoint `make_get_ai_payload` with `job_id`.
4. Filter: job status `queued|retry_wait`, entity current, payload hash matches.
5. HTTP POST `https://api.openai.com/v1/responses`, or the current Make OpenAI module if it can pass the full Responses API payload without losing Structured Outputs.
6. Parse the JSON Structured Output.
7. Validate: schema version, source keys, forbidden claims, numbers unchanged.
8. HTTP POST Bubble `make_complete_ai_job`.
9. Webhook response/finish.

The result is not published directly: Bubble creates an AI Analysis with `review_status = pending`.

Error route:

- 429/connection/timeout → Retry handler + incomplete execution;
- invalid model JSON → one regeneration with the same input and a repair instruction; then `failed_validation`;
- Bubble 409 stale snapshot → mark the job `cancelled_stale`, no retry;
- OpenAI policy/refusal → `failed_refusal`, show the admin the deterministic fallback;
- callback failure → retry the callback; do not repeat the OpenAI call if the provider result is already stored in the execution bundle.

MK-02 Transactional email dispatcher

Events: email verification, password reset (if not handled by Bubble), application received, verification decision, changes requested, project decision, enquiry received, status update.

The payload contains `notification_id` and `template_key`, not arbitrary HTML. Make fetches approved template data from Bubble.

Steps:

1. Receive job.
2. Fetch Notification payload.
3. Router by `template_key`.
4. Send through the email provider.
5. Callback with the provider message id.

Rules:

- do not mix transactional and marketing email;
- send marketing messages only with an active Consent Record;
- unsubscribe does not apply to security/transactional messages;

- record template version and language on the Notification.

MK-03 Approved introduction delivery

This is a high-risk flow because it discloses personal data to both parties.

Preconditions in Bubble before the job is created:

- Enquiry status `approved_for_intro`;
- the investor has active consent to contact sharing;
- Unit/Project/Company are not suspended;
- a Contact Release exists and contains the field `whitelist`;
- admin confirmation is complete.

Make steps:

1. Fetch the one-time introduction payload.
2. Send the investor email with the whitelisted developer contact.
3. Send the developer email with the whitelisted investor contact.
4. If one delivery succeeds and the other fails, do not resend the first; store the provider id on the channel item and retry only the failed leg.
5. Callback with the results of both deliveries.
6. Bubble moves `approved_for_intro` → `introduced` only when the required deliveries have succeeded.

Channel-level idempotency:

```
intro:<enquiry_id>:investor:v1
intro:<enquiry_id>:developer:v1
```

MK-04 Decision notification

Events:

- developer application `more_info_required|approved|rejected`;
- project `changes_requested|approved|published|rejected`;
- change request `queried|approved|rejected|applied`;
- enquiry `on_hold` (no emails) and `declined`.

Make delivers the notification; the business transition has already happened in Bubble. An email failure does not roll back the decision but creates an admin alert and a retry job.

Rules for a declined introduction.

- `on_hold` — **no email** to either the investor or the developer. If the scenario receives a job for this status with an email template, that is a configuration error: the job must end as `failed_validation`, not as a send.
- `declined` — **exactly one** email, to the investor, using a neutral template: the property is not available, plus three similar properties. The decline reason **does not** go into the email and is **not passed** into the Make payload at all.
- The developer receives no email on a decline; they see only an aggregated count of filtered-out requests in their portal.

This means the payload for `declined` must contain neither `reason` nor `note_private` — otherwise the reason would end up in Make logs.

MK-05 Price and availability alerts

Triggered after a Change Request is successfully applied and financials/score are recalculated.

Because both price and availability go through admin approval, this scenario always starts from one point — after `apply_change_request`. There is no separate branch for an "immediate" availability change.

Bubble prepares a list of Notification IDs for:

- investors who saved the Unit;
- investors with an open Enquiry;
- saved searches that stopped/started matching.

Make does not run broad searches in the Bubble Data API. The fan-out is built by backend workflows with batching.

Events:

- price changed by a configured threshold;
- `available` → `reserved|sold`;
- → `withdrawn` (admin action, not the developer's);
- listing back to available;
- score verdict changed.

MK-06 Saved-search match digest

Schedule: daily at 08:00 in the user's timezone, or one global UTC batch in the MVP.

The Bubble endpoint returns a batch of Notification IDs already built according to privacy/business rules. Make sends the digest and the callback. No per-user uncontrolled Data API scans in Make.

MK-07 Integration dead-letter alert

Schedule: every 15 minutes.

1. Query Bubble for `Integration Job` with status `failed|dead_letter` or overdue `processing`.
2. Group by job type/error code.
3. Send an operational alert to the responsible admins.
4. Do not include PII/payload in the subject or chat notification.

The alert contains the job id, entity public id, environment, attempts, redacted error and a direct admin URL.

MK-09 Bulk re-score progress relay

Trigger: Integration Job `job_type = score_rescore_batch`, created after the admin saves a new Score Model Version.

The Score Editor screen shows progress ("N of 83"), a completion state with a rollback option, and a failure state with a reference and the buttons "Retry the remainder" / "Roll back". For this to work without the browser polling Bubble, a scenario is needed that reports progress.

Steps:

1. Bubble itself runs the recalculation in batches in a backend workflow (calculation stays in Bubble — Make does not compute the score).
2. After each batch Bubble updates `processed_count` on the Integration Job.
3. MK-09 sends an operational notification only in two cases: full completion, and final failure after retries are exhausted.
4. Intermediate progress is read by the UI directly from the Integration Job.

Rules:

- a recalculation **never** overwrites historical Listing Scores — new records are created, old ones get `is_current = no` (BR-04);
- rolling back to a previous model version is a new batch job, not a deletion;
- a partial failure leaves the catalogue in a mixed state, so the alert is P2 and the message must state how many properties were processed.

MK-10 Comparable rental data collection

Trigger: schedule. The frequency depends on each provider's terms and is set in the Data Provider reference table.

The market rent estimate is built from collected comparable listings, not entered manually by an analyst.

Steps:

1. Get from Bubble the list of active Data Providers and the parameter sets that need a fresh sample. "Needs" means the current sample is older than `Country Config.max_comparable_age_days` (default **90**) or does not exist. Bubble builds the list — Make does not decide on its own what is stale.
2. For each set, call the provider's official API or feed.
3. Normalise the response by country, city/district, property type, number of bedrooms and floor area.
4. Compute `listings_count`, `rent_min`, `rent_median`, `rent_max`.
5. Send the result to Bubble in one callback — a **Rental Comparable Set** is created with `review_status = pending`.

Rules:

- Make does **not** write the result into a Financial Input and does not change any published figure. It creates a sample for the admin to review;
- a sample with `listings_count` below `Country Config.min_comparable_listings` (default **5**) is still stored but marked insufficient and cannot be approved — Make does not discard it; Bubble makes the decision;
- a sample between the minimum and `sufficient_comparable_listings` (default **10**) is stored with a **thin sample** flag;
- a new sample does not overwrite the previous one (Bubble switches `is_current`);

- `collection_method` records how the data was obtained — `api`, `feed`, or `manual_import` for the fallback;
- rate limits and terms of use are configured per provider; hitting a limit is not an error but a reason to lower the schedule frequency;
- if a provider is unavailable for several cycles in a row, an operational alert is raised: otherwise the rent estimate would go stale silently.

Fallback without an API. If a portal has no permitted API or feed, the sample is entered manually through the Bubble admin. Make is not involved in that case, but the record is created with the same type and the same fields.

Legal note. Connecting each source requires a check of its data terms of use. This is a contractual matter, not a technical one — see `terms_reference` on the Data Provider.

7. Idempotency and concurrency

Rules for every scenario:

- Bubble creates the `idempotency_key`; Make never generates it;
- before a side effect, Make checks the current Job status;
- after a side effect, it stores the provider id and returns it in the callback;
- a repeated bundle with the same key does not resend the email/AI request if a provider id already exists;
- for Unit change jobs, the key includes the target version;
- for webhook scenarios where order matters, enable **Process data in order**;
- independent email jobs may run in parallel within the provider rate limit.

Make webhooks are processed in parallel by default, so ordering cannot be assumed without the corresponding scenario setting.

8. Error handling policy

In all production scenarios:

- `Store incomplete executions = Yes`;
- automatic retry for connection/rate-limit/timeouts;
- Retry error handler for important external modules;
- no silent Ignore/Skip for AI, introductions, decisions or transactional email;
- invalid business data → callback `failed_validation`, no endless retries;
- temporary failure → exponential/backoff retry;
- permanent 4xx authentication/configuration failure → dead letter + alert;
- 3–5 attempts depending on the provider, then manual resolution;
- `Discard data if storage is full = No` for critical scenarios;
- do not enable `Commit after each module` without a specific need; control side effects with your own channel-level idempotency.

Make incomplete executions are a recovery mechanism, not long-term storage. Critical data stays in the Bubble Integration Job/Notification.

9. Data Store in Make

Allowed:

- short-lived dedup/lock by idempotency key, if a scenario module requires it;
- cache of non-sensitive reference data;
- cursor for a technical scheduled batch.

Forbidden:

- master records for Users, Projects, Units, Enquiries;
- copies of verification documents;
- long-term storage of contact data;
- being the only source of delivery status.

If a Data Store is used for dedup, record key = idempotency key, TTL is cleared by scheduled maintenance, and the authoritative status is still checked in Bubble.

10. Logs and confidentiality

- Do not pass more data into Make scenario logs than necessary.
- For introductions and PII-heavy flows, consider `Keep data confidential`, bearing in mind it limits debugging; the choice must be aligned with the incident procedure.
- Error messages returned to Bubble are redacted: no token, email, document URL or raw OpenAI prompt.
- Correlation ID = Integration Job public_id in Bubble, Make and external metadata.
- Retention of Make logs/incomplete executions is documented separately as a subprocessor setting.

11. Monitoring and SLA

Dashboard metrics:

Metric	MVP target
Webhook acceptance p95	< 2 s
AI job complete p95	< 60 s
Transactional email queued p95	< 2 min
Introduction complete p95	< 5 min
Failed jobs after retries	< 1%
Duplicate side effects	0
Jobs stuck processing > 15 min	0

Alert levels:

- P1: introduction delivered to one party only; possible data leak; credential compromise.
- P2: AI/transactional scenario dead-letter, >5 consecutive failures, provider auth error.
- P3: digest delay, individual non-critical email failure.

12. Naming convention in Make

```
BIP | PROD | MK-01 | Generate investment analysis
BIP | DEV  | MK-01 | Generate investment analysis
```

Webhook names, connections and data stores must include the environment. Scenario notes must include owner, purpose, payload version, Bubble endpoints, retry policy and last review date.

13. Test scenarios

Must be verified:

1. valid AI job → pending-review AI Analysis;
2. duplicate webhook → one AI call / one result;
3. OpenAI 429 → retry without a duplicate callback;
4. stale score version → cancel without publication;
5. malformed structured output → one repair attempt → failure;
6. approved introduction → two unique emails and status `introduced`;
7. failure of the second introduction email → retry only the second;
8. invalid/missing consent → no job is created;
9. Bubble callback timeout after external success → retry the callback, not the side effect;
10. Make queue/rate-limit response → the Bubble job stays retryable;
11. email opt-out → the marketing digest is not sent; transactional email is sent per the rules;
12. private data is not visible in a non-admin Make alert;
13. enquiry `on_hold` → no email; a job with an email template for this status ends as `failed_validation`;
14. enquiry `declined` → exactly one email to the investor; the reason is absent from the email body, the payload and the Make logs;
15. an availability change without an approved Change Request does not create an alert job;
16. a score recalculation with a partial failure leaves historical Listing Scores untouched and raises a P2 alert with the processed count;
17. MK-10 creates a Rental Comparable Set with `review_status = pending` and does not change any Financial Input;
18. a sample with `listings_count = 4` against a minimum of 5 is stored but cannot be approved; a sample of 7 is approved with a thin sample flag; a sample older than 90 days is not offered as a source;
19. a provider being unavailable for several cycles in a row raises an alert rather than letting the estimate go stale silently;
20. re-running MK-10 with the same parameters does not create a duplicate of the current sample.

14. Make readiness criteria

- all production webhooks are protected by an API key/secret and have a versioned schema;
- all critical scenarios have incomplete executions, a retry route and a dead-letter alert;
- secrets are separated Dev/Live and never appear in payload/URL/log text;

- every scenario has a documented owner and a rollback/disable procedure;
- a manual replay does not produce a duplicate email, AI analysis or status event;
- the Bubble admin sees the status, attempts and redacted error for every job;
- financial/approval decisions do not depend on the Make Data Store;
- introduction PII is passed only after explicit Bubble authorisation and consent validation.